

FlowBoard — Specifica di creazione

Seconda applicazione Angular per lo studio sulla robustezza dei locatori (tesi) · brief per la generazione automatica · 2026-07-10

1. Scopo e contesto

FlowBoard è una **Kanban board** in Angular 19, seconda applicazione di prova (dopo CineLib) per il requisito di **generalità** della tesi. Serve a generare mutanti del markup e misurare quanto le **6 strategie di locatori** (Absolute, Relative, Selenium, Katalon, Robula, Robula+) resistono alle mutazioni.

Obiettivo specifico che distingue questa app da CineLib: deve essere progettata perché **tutti e 11 gli operatori di mutazione (a-k) producano mutanti VALIDI (compilabili) e misurabili**, incluso l'operatore **h** (spostamento di un elemento tra due template di componenti diversi), che su CineLib era not-compilable per costruzione.

2. LA REGOLA D'ORO (perché h falliva e come evitarlo)

L'operatore **h** sposta un elemento dal template di un componente a quello di un **altro** componente. Su CineLib gli elementi-target erano legati a **dati locali del componente** (es. `{{ movie().title }}`, `{{ member().name }}`, `[member]="member"`): spostati altrove, quei binding puntano a variabili inesistenti → **non compila**.

Regola d'oro: gli elementi che fanno da bersaglio ai test devono essere **(A) statici** (nessun binding) **oppure (B) legati a uno store globale a signal** iniettato in TUTTI i componenti. Così un elemento come `{{ store.selected()?.title }}` compila in qualsiasi componente → **h produce mutanti validi**.

Vietato il "prop drilling" con `@Input` per gli elementi bersaglio: usare sempre lo store globale.

3. Stack tecnologico e vincoli

- **Angular 19**, componenti **standalone**, **signals**, control flow `@if / @for / @empty`.
- **Nessun backend:** dati 100% locali. Niente HTTP, niente OAuth, niente API esterne, niente rate-limit.
- **Deterministico:** seed di dati **fisso e hard-coded**. Nessun valore casuale, nessuna data/ora dinamica (`Date.now()`, date correnti): se servono date, usare stringhe fisse nel seed.
- **Reset:** l'app parte sempre dallo stesso seed; un pulsante "Reset board" ripristina lo stato iniziale. I test partono da stato pulito (come CineLib). Se si usa `localStorage`, dev'essere azzerabile e i test lo svuotano prima di ogni scenario.
- Router Angular per le pagine; nessun login.

4. Architettura: un unico store globale a signal

Creare un servizio `BoardStore` (`providedIn: 'root'`) che contiene **tutto** lo stato in signals. Tutti i componenti leggono da qui.

- Stato: `columns()`, `cards()`, `selectedCard()`, `filters()` (testo/priorità/assegnatario), `totalCards()`, conteggi per colonna e per priorità.
- Derivati (computed): `cardsByColumn(colId)`, `filteredCards()`, `backlogCount()`, `doneCount()`, ecc.
- Azioni: `addCard`, `updateCard`, `removeCard`, `moveCard(id, colId)`, `select(id)`, `setFilter`, `reset`.

5. Struttura: route e componenti

Route	Componente	Contenuto
/	app-board	toolbar (ricerca, filtro, add, reset) + 4 colonne + card
/card/:id	app-card-detail	dettaglio di <code>store.selectedCard()</code> (single, store-bound)
/card/new e /card/:id/edit	app-card-form	form (titolo, descrizione, priorità, assegnatario, colonna, tag)
/stats	app-stats	aggregati store-bound + tabella

Componenti aggiuntivi: `app-header` (barra in alto persistente), `app-column`, `app-card` (card nella lista), `app-toast`.

Scelta chiave per h: le **4 colonne sono esplicite** (Backlog, In Progress, Review, Done), **non generate in @for**: così i titoli di colonna sono **statici** e i conteggi vengono dallo store (`store.backlogCount()`) → questi elementi sono **h-safe**. Le **card dentro le colonne** restano in `@for` (per gli altri operatori).

6. Modello dati e seed deterministico

- **Column:** `id`, `title` (fisso: Backlog/In Progress/Review/Done).
- **Card:** `id` (numerico fisso), `title`, `description`, `priority` ('low'|'medium'|'high'), `assignee`, `tags` (array), `columnId`.
- **Seed fisso:** ~10 card distribuite nelle 4 colonne, con id e titoli stabili (es. #1 "Setup repository" in Backlog, #3 "Design login" in In Progress, ecc.). Nessuna casualità.

7. Attributi `x-test-*` e multi-strategia

Come su Cinelib: **ogni elemento interattivo o bersaglio deve avere un attributo `x-test-*` univoco** (la "ground truth", **mai** da mutare). In più, ogni bersaglio deve essere raggiungibile con **più strategie**: classe CSS, testo, ruolo ARIA, attributo `data-*` (es. `data-priority`, `data-column`), e posizione gerarchica. Così si possono autorare tutte e 6 le strategie di locatori.

Esempi: `x-test-board`, `x-test-column`, `x-test-card`, `x-test-card-title`, `x-test-add-card`, `x-test-search`, `x-test-filter-priority`, `x-test-reset`, `x-test-total-badge`, `x-test-detail-title`, `x-test-form-submit`, `x-test-stat-total`, ecc.

8. Come rendere applicabili TUTTI gli 11 operatori

L'app deve contenere, in abbondanza, gli "ingredienti" che ogni operatore richiede. In particolare per **h** servono molti elementi **statici** o **store-bound**.

Op	Tipo di mutazione	Cosa deve esistere nell'app
a	Attribute Value Modification	elementi con attributi valorizzati (<code>data-*</code> , <code>aria-*</code> , <code>class</code> , <code>title</code>)
b	Attribute Removal	attributi non essenziali removibili (stessi di sopra)
c	Attribute Identifier Modification	attributi rinominabili senza rompere Angular (preferire <code>data-*</code> / <code>aria-*</code> , NON binding)
d	Text Content Modification	elementi con testo visibile (titoli, etichette, badge)
e	Text Content Removal	idem
f	Tag Movement (nel contenitore)	contenitori con più figli fratelli (toolbar, riga di badge, form-actions)
g	Tag Movement (qualsiasi punto)	alberi annidati con più livelli
h	Tag Movement (tra template)	molti elementi STATICI (badge, icone, banner, etichette, titoli statici di colonna) e STORE-BOUND (contatore totale, conteggi colonna, dettaglio da <code>store.selected()</code>) → compilano se spostati altrove
i	Tag Removal (unwrap)	elementi wrapper con figli/testo (div che avvolge span/testo)
j	Tag Type Modification	elementi con tag variabili (div/section/article/span intercambiabili)
k	Tag Insertion	elementi attorno a cui si può inserire un wrapper

Zone h-safe da prevedere (elementi statici o store-bound, fuori dai `@for`):

- **Header:** titolo app (statico), badge totale card (`store.totalCards()`), pulsante "New card" (etichetta statica).
- **Toolbar:** etichette statiche, conteggi da store, banner statici ("Drag to reorder", "Board demo").
- **Titoli e conteggi di colonna** (colonne esplicite, non in `@for`).
- **Card detail** (`/card/:id`): tutti gli elementi da `store.selectedCard()` .
- **Stats** (`/stats`): stat card store-bound + intestazioni statiche + tabella.
- Icone, etichette, messaggi di stato vuoto (`@empty`), footer — tutti statici.

Le **card dentro le colonne** (in `@for`, legate alla variabile di ciclo) restano bersaglio per gli operatori a–e, f, g, i, j, k, ma NON per h.

9. Scenari E2E con condizioni di pre/post successo

Ogni scenario ha uno **stato iniziale noto** (pre), un'**azione**, e uno **stato atteso** (post), verificabile con i locatori. Esempi deterministici:

Scenario	Pre-condizione	Azione	Post-condizione
Ricerca card	10 card nel board	digita "login" nella ricerca	1 card visibile, titolo "Design login"

Filtro priorità	badge totale = 10	filtro "High"	N card con <code>data-priority="high"</code>
Sposta card	card #1 in Backlog	sposta in "In Progress"	conteggio Backlog -1, In Progress +1
Aggiungi card	form vuoto su <code>/card/new</code>	compila e submit	badge totale = 11, card in colonna scelta
Dettaglio	board	apri card #3	titolo dettaglio = "Design login"
Elimina	card #5 esistente	elimina	badge totale = 9, card assente

I test devono partire dal seed (chiamare `reset` / svuotare `localStorage` in fase di setup), come su CineLib.

10. Anti-pattern da EVITARE

- **NO** `@Input` /prop drilling per gli elementi bersaglio: usare lo store globale (altrimenti h si rompe).
- **NO** dati casuali, id generati a runtime, date/ore correnti: rompono il determinismo dei test.
- **NO** backend, HTTP, autenticazione, servizi esterni.
- **NON** mettere logica solo dentro `@for` : prevedere abbastanza elementi statici/store-bound per h.
- **NON** mutare mai gli attributi `x-test-*` (sono la ground truth): l'app li tiene, il generatore non li tocca.
- **NO** animazioni/timeout che rendano instabile lo stato durante i test.

11. Deliverable atteso da Fable

- App Angular 19 funzionante (`npm start`), avviabile su una porta configurabile (userei **4400** per non confliggere con CineLib 4300 e Spotify 4200).
- Store globale `BoardStore` a signal; nessun backend; seed deterministico + `reset`.
- Tutti gli elementi bersaglio con `x-test-*` e raggiungibili con più strategie (classe/testo/ruolo/data-*/posizione).
- Zone h-safe (statiche + store-bound) come da §8.
- Un `README` che elenca le route, gli attributi `x-test-*` principali e gli scenari E2E con pre/post condizioni.
- (Opzionale) una manciata di test Playwright d'esempio come su CineLib, per fissare gli scenari.

Nota: lo scopo NON è un prodotto reale, ma un banco di prova ricco e deterministico dove ogni tipo di mutazione (a-k) sia applicabile e produca mutanti compilabili, per un confronto equo e completo delle 6 strategie di locatori.